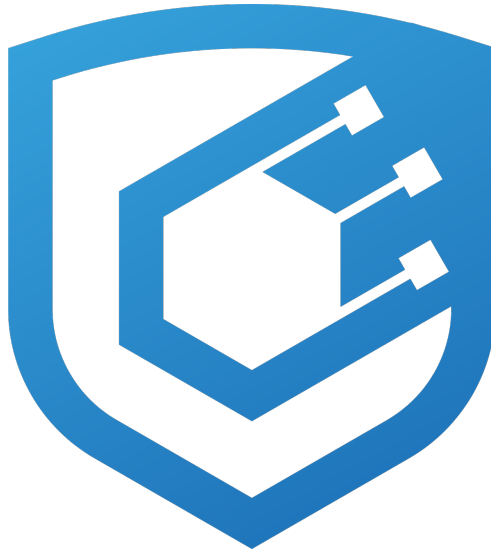




TSwap Protocol

Security Audit Report

Version 1.0

Prepared by: Farhan Davin (Lead Auditor)

September 28, 2026

TSwap Protocol Security Audit Report

Farhan Davin

September 28, 2026

Prepared by: Farhan Davin (Lead Security Researcher)

Audited Codebase: Cyfrin / 5-t-swap-audit

Commit Hash: e643a8d4c2c802490976b538dd009b351b1c8dda

Methodology: The Tincho Method

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
 - High Severity Findings
 - * [H-1] Unaccounted Swap Incentive Reward in `_swap` Breaks Constant Product Invariant ($x \cdot y = k$) and Enables Liquidity Reserve Draining
 - * [H-2] Incorrect Numerator Constant (10000 vs 1000) in `getInputAmountBasedOnOutput` Overcharges Traders by ~10x

- * [H-3] Inverted Call in `sellPoolTokens` Passes Input Amount as Desired Output, Causing Catastrophic Token Loss or Revert
- * [H-4] Hardcoded 18-Decimal Swap Reward in `_swap` Permanently Blocks All Swaps for Non-18 Decimal Token Pools (USDC, WBTC)
- Medium Severity Findings
 - * [M-1] Absence of `maxInputAmount` Slippage Protection in `swapExactOutput` Exposes Users to Extreme Capital Theft via Sandwich Attacks
 - * [M-2] Unused `deadline` Parameter in `deposit` Fails to Enforce Transaction Expiration, Enabling Execution of Stale Mempool Transactions
 - * [M-3] Unassigned Named Return Variable in `swapExactInput` Always Evaluates to Zero, Breaking Composability
- Low Severity Findings
 - * [L-1] Swapped Arguments in `LiquidityAdded` Event Emission Corrupts Off-Chain Accounting
 - * [L-2] `PoolFactory.createPool` Permits WETH/WETH Pairs and Zero Address Token Deployments
 - * [L-3] `PoolFactory.createPool` Concatenates Token Name Instead of Symbol for LP Token Symbol
- Informational Findings
 - * [I-1] Missing Zero Address Validation in `PoolFactory` Constructor Risks Permanent Misconfiguration
 - * [I-2] Unused Custom Error `PoolFactory__PoolDoesNotExist` Increases Bytecode Size
 - * [I-3] Unused Storage Read `poolTokenReserves` in `deposit` Wastes Gas (Compiler Warning 2072)
 - * [I-4] Unindexed Event Parameters in `PoolFactory.PoolCreated` and `TSwapPool.Swap` Prevent Efficient Log Filtering
 - * [I-5] Strict Equality Comparison in `revertIfZero` Modifier
 - * [I-6] Erroneous Mathematical Explanation in NatSpec / Comments Misleads Developers
 - * [I-7] Redundant Wrapper Function `totalLiquidityTokenSupply()` Adds Bytecode Overhead
 - * [I-8] Unnamed Magic Numbers Obscure Business Logic and Contribute to Math Errors
 - * [I-9] Private Function Declarations Preceding Public View Functions Violates Solidity Style Guide
 - * [I-10] License Identifier Discrepancy Between Protocol (GPL-3.0) and Depend-

cies (MIT)

* [I-11] Unused Function Parameter `deadline` in `deposit` Triggers Compiler Warning (5667)

- Test Suite Quality & Mutation Testing Review
- General Recommendations

Protocol Summary

TSwap is an Automated Market Maker (AMM) protocol loosely modeled after Uniswap v1. It facilitates permissionless token swaps between arbitrary ERC20 tokens and Wrapped Ether (WETH).

The protocol operates via a core factory contract, `PoolFactory`, which handles the permissionless deployment and registry of individual token pools (`TSwapPool`). Each pool maintains balances of two assets: a single ERC20 token and WETH.

Liquidity providers deposit both assets in equal proportion to the current reserve ratio and receive ERC20 LP shares representing their proportional claims on the underlying reserves and accumulated 0.3% trading fees. Traders interact with each pool via `swapExactInput` and `swapExactOutput` functions, with prices dictated by the constant product formula:

$$(x + \Delta x \cdot (1 - \rho)) \cdot (y - \Delta y) \geq x \cdot y$$

where $\rho = 0.003$ (representing the 0.3% trading fee).

Disclaimer

The audit team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit is not an endorsement of the underlying business or product. The review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

Likelihood	Impact	High	Medium	Low
High		High	High/Medium	Medium
Medium		High/Medium	Medium	Medium/Low
Low		Medium	Medium/Low	Low

We use the CodeHawks severity matrix to determine severity based on Impact and Likelihood.

Audit Details

Scope

- **Repository:** Cyfrin / 5-t-swap-audit
- **Commit Hash:** [e643a8d4c2c802490976b538dd009b351b1c8dda](#)
- **Solidity Version:** 0.8.20

In-Scope Contracts

```
1 src/  
2 |-- PoolFactory.sol      (64 lines, Registry and pair deployment)  
3 #-- TSwapPool.sol       (429 lines, Core AMM pricing and liquidity  
    logic)
```

Roles

- **Liquidity Provider (LP):** Unprivileged actor depositing dual assets to earn pro-rata swap fees.
- **Trader (Swapper):** Unprivileged actor executing swaps via exact-input or exact-output paths.
- **Factory Deployer:** Unprivileged initial deployer of `PoolFactory`.

Executive Summary

During September 2026, a security review of the **TSwap** protocol was conducted using **The Tincho Method**. The audit identified **4 High Severity**, **3 Medium Severity**, **3 Low Severity**, and **11 Informational** vulnerabilities.

Multiple critical vulnerabilities directly violate core economic invariants, allow draining of liquidity reserves, cause massive pricing overcharges (~10x), and trigger complete Denial of Service for common non-18 decimal ERC20 tokens like USDC.

Issues found

Severity	Number of issues found	Resolved	Acknowledged
High	4	0	4
Medium	3	0	3
Low	3	0	3
Informational	11	0	11
Total	21	0	21

Findings

High Severity Findings

[H-1] Unaccounted Swap Incentive Reward in `_swap` Breaks Constant Product Invariant ($x \cdot y = k$) and Enables Liquidity Reserve Draining

Description:

In `TSwapPool.sol`, every 10 swaps (`SWAP_COUNT_MAX`), the contract awards the trader an additional `1e18` (1 token) of `outputToken` directly from pool reserves:

```
1 swap_count++;
2 if (swap_count >= SWAP_COUNT_MAX) {
3     swap_count = 0;
4     outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000);
5 }
```

5 }

This transfer is executed from the pool's core asset reserves without receiving any corresponding input tokens or adjusting invariant accounting.

Impact:

1. **Protocol Invariant Violation:** Directly breaks $(x + \Delta x) \cdot (y - \Delta y) \geq x \cdot y$. Pool reserves decrease without any price compensation, causing the invariant k to decay over time. This is the exact root cause triggering failure in stateful fuzz tests (`statefulFuzz_constantProductFormulaStaysTheSameX`).
2. **Liquidity Drain:** An attacker can submit 9 micro-swaps (e.g. 10 wei of input tokens) and on the 10th swap extract an entire `1e18` of WETH or Pool Token, effectively draining honest liquidity providers' capital.
3. **MEV Exploitation:** Front-running bots can monitor the mempool when `swap_count == 9` and siphon the reward before legitimate users interact with the pool.

Proof of Concept:

An attacker executes 9 dust trades (100 wei each) and extracts an extra 1 WETH on the 10th swap:

```

1 function test_PoC_InvariantBrokenAndReservesDrainedBySwapIncentive
  () public {
2     vm.startPrank(attacker);
3     poolToken.approve(address(pool), type(uint256).max);
4
5     uint256 attackerWethBefore = weth.balanceOf(attacker);
6     uint256 poolWethBefore = weth.balanceOf(address(pool));
7
8     // Attacker executes 9 minimal dust trades
9     for (uint256 i = 0; i < 9; i++) {
10         pool.swapExactInput(poolToken, 100, weth, 0, uint64(block.
            timestamp));
11     }
12
13     // On the 10th swap, attacker extracts standard output + 1e18
        free WETH
14     pool.swapExactInput(poolToken, 100, weth, 0, uint64(block.
        timestamp));
15     vm.stopPrank();
16
17     uint256 attackerWethAfter = weth.balanceOf(attacker);
18     uint256 poolWethAfter = weth.balanceOf(address(pool));
19
20     // Attacker extracted > 1 WETH while spending negligible dust
21     assertGt(attackerWethAfter - attackerWethBefore, 1e18);
22     assertGt(poolWethBefore - poolWethAfter, 1e18);

```

```
23 }
```

Recommended Mitigation:

Remove the arbitrary swap reward mechanism from core AMM swap logic. If trader incentives are required, handle them through a dedicated external rewards contract funded separately:

```
1 --- a/src/TSwapPool.sol
2 +++ b/src/TSwapPool.sol
3 @@ -43,8 +43,6 @@ contract TSwapPool is ERC20 {
4     IERC20 private immutable i_wethToken;
5     IERC20 private immutable i_poolToken;
6     uint256 private constant MINIMUM_WETH_LIQUIDITY = 1
7         _000_000_000;
8     uint256 private swap_count = 0;
9     uint256 private constant SWAP_COUNT_MAX = 10;
10 @@ -396,11 +394,6 @@ contract TSwapPool is ERC20 {
11     revert TSwapPool__InvalidToken();
12 }
13
14 - swap_count++;
15 - if (swap_count >= SWAP_COUNT_MAX) {
16 -     swap_count = 0;
17 -     outputToken.safeTransfer(msg.sender, 1
18 -         _000_000_000_000_000_000);
19 - }
20 emit Swap(
```

[H-2] Incorrect Numerator Constant (10000 vs 1000) in getInputAmountBasedOnOutput Overcharges Traders by ~10x**Description:**

In `TSwapPool.sol`, the pricing formula in `getInputAmountBasedOnOutput()` calculates the input tokens required to receive a given `outputAmount`:

```
1 return
2     ((inputReserves * outputAmount) * 10000) /
3     ((outputReserves - outputAmount) * 997);
```

Under standard constant product AMM math with a 0.3% trading fee, the numerator multiplier must be 1000, not 10000.

Impact:

Every trader calling `swapExactOutput()` or using `sellPoolTokens()` is overcharged by a factor of approximately $10\times$. For example, purchasing 10 WETH in a 100/100 pool costs ≈ 111.4 tokens instead of the mathematically correct ≈ 11.14 tokens, causing immediate financial loss to traders.

Proof of Concept:

```
1 function test_PoC_TenXFeeCalculationOvercharge() public view {
2     uint256 inputReserves = 100e18;
3     uint256 outputReserves = 100e18;
4     uint256 outputAmount = 10e18;
5
6     // Expected input under Uniswap CPAMM with 0.3% fee:
7     uint256 expectedInputAmount = ((inputReserves * outputAmount *
8         1000) /
9         ((outputReserves - outputAmount) * 997)) + 1;
10
11    // TSwap actual calculated input:
12    uint256 actualInputAmount = pool.getInputAmountBasedOnOutput(
13        outputAmount, inputReserves, outputReserves);
14
15    // Actual input is ~10x greater than mathematically intended
16    assertGt(actualInputAmount, expectedInputAmount * 9);
17 }
```

Recommended Mitigation:

Correct the numerator constant from 10000 to 1000 and add + 1 to round up in favor of pool solvency:

```
1 --- a/src/TSwapPool.sol
2 +++ b/src/TSwapPool.sol
3 @@ -291,7 +291,7 @@ contract TSwapPool is ERC20 {
4     returns (uint256 inputAmount)
5     {
6         return
7 -         ((inputReserves * outputAmount) * 10000) /
8 -         ((outputReserves - outputAmount) * 997);
9 +         (((inputReserves * outputAmount) * 1000) /
10 +         ((outputReserves - outputAmount) * 997)) + 1;
11     }
```

[H-3] Inverted Call in sellPoolTokens Passes Input Amount as Desired Output, Causing Catastrophic Token Loss or Revert**Description:**

The NatSpec documentation for `sellPoolTokens()` describes its purpose as allowing users to sell a specified `poolTokenAmount` of pool tokens to receive WETH:

```
1 function sellPoolTokens(  
2     uint256 poolTokenAmount  
3 ) external returns (uint256 wethAmount) {  
4     return  
5         swapExactOutput(  
6             i_poolToken,  
7             i_wethToken,  
8             poolTokenAmount,  
9             uint64(block.timestamp)  
10        );  
11 }
```

However, `swapExactOutput()` treats its 3rd argument as `outputAmount`. Passing `poolTokenAmount` causes the function to interpret the value as the desired *WETH output*, rather than the pool tokens being sold.

Impact:

When a user attempts to sell 10 Pool Tokens (`10e18`), the contract interprets this as “user wants to receive 10 WETH output”. Combined with the 10x fee error, the user is charged ≈ 111.4 Pool Tokens instead of 10. If the user lacks sufficient tokens or approval, the transaction reverts; if they possess them, they suffer catastrophic and unintended token deductions.

Proof of Concept:

```
1 function test_PoC_SellPoolTokensInvertedLogic() public {  
2     poolToken.mint(user, 200e18);  
3     vm.startPrank(user);  
4     poolToken.approve(address(pool), type(uint256).max);  
5  
6     uint256 poolTokenBalanceBefore = poolToken.balanceOf(user);  
7     // User intends to sell 10 pool tokens  
8     pool.sellPoolTokens(10e18);  
9     uint256 poolTokenBalanceAfter = poolToken.balanceOf(user);  
10  
11     uint256 tokensDeducted = poolTokenBalanceBefore -  
12         poolTokenBalanceAfter;  
13     // Over 111 pool tokens were deducted instead of 10  
14     assertGt(tokensDeducted, 100e18);  
15     vm.stopPrank();  
16 }
```

Recommended Mitigation:

Rewrite `sellPoolTokens` to invoke `swapExactInput()` rather than `swapExactOutput()`, and include slippage protection:

```

1 --- a/src/TSwapPool.sol
2 +++ b/src/TSwapPool.sol
3 @@ -363,13 +363,14 @@ contract TSwapPool is ERC20 {
4     * @return wethAmount amount of WETH received by caller
5     */
6     function sellPoolTokens(
7 -         uint256 poolTokenAmount
8 +         uint256 poolTokenAmount,
9 +         uint256 minWethToReceive,
10 +         uint64 deadline
11     ) external returns (uint256 wethAmount) {
12         return
13 -         swapExactOutput(
14 +         swapExactInput(
15             i_poolToken,
16 -         i_wethToken,
17             poolTokenAmount,
18 -         uint64(block.timestamp)
19 +         i_wethToken,
20 +         minWethToReceive,
21 +         deadline
22     );
23     }

```

[H-4] Hardcoded 18-Decimal Swap Reward in `_swap` Permanently Blocks All Swaps for Non-18 Decimal Token Pools (USDC, WBTC)

Description:

In `TSwapPool.sol`, `_swap()` attempts to transfer a hardcoded reward of `1_000_000_000_000_000_000` units of `outputToken`:

```
1 outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000);
```

While this corresponds to 1 token for an 18-decimal token, for tokens with fewer decimals (e.g. USDC with 6 decimals or WBTC with 8 decimals), this represents an astronomical quantity:

- USDC (6 decimals): `1e18` units = 1,000,000,000,000 (1 Trillion) USDC.

Impact:

When a pool paired with a non-18 decimal token reaches 10 swaps where the non-18 decimal token is the `outputToken`, `safeTransfer()` attempts to transfer 1 trillion tokens. Because the pool reserves are vastly lower than 1 trillion tokens, the ERC20 transfer reverts due to insufficient balance. Because `swap_count` is 9, every subsequent swap attempting to purchase the token will

hit this revert condition, causing permanent Denial of Service (DoS) and trapping remaining liquidity.

Proof of Concept:

```
1 function test_PoC_Non18DecimalTokenCausesPermanentDoS() public {
2     MockUSDC usdc = new MockUSDC(); // 6 decimals
3     TSwapPool usdcPool = TSwapPool(factory.createPool(address(usdc)
4         ));
5     usdc.mint(liquidityProvider, 10_000 * 1e6);
6     weth.mint(liquidityProvider, 10e18);
7
8     vm.startPrank(liquidityProvider);
9     usdc.approve(address(usdcPool), type(uint256).max);
10    weth.approve(address(usdcPool), type(uint256).max);
11    usdcPool.deposit(10e18, 10e18, 10_000 * 1e6, uint64(block.
12        timestamp));
13    vm.stopPrank();
14
15    weth.mint(user, 10e18);
16    vm.startPrank(user);
17    weth.approve(address(usdcPool), type(uint256).max);
18
19    // First 9 swaps succeed
20    for (uint256 i = 0; i < 9; i++) {
21        usdcPool.swapExactInput(weth, 100, usdc, 0, uint64(block.
22            timestamp));
23    }
24
25    // 10th swap attempts to transfer 1e18 USDC (1 trillion units)
26    // and reverts permanently
27    vm.expectRevert();
28    usdcPool.swapExactInput(weth, 100, usdc, 0, uint64(block.
29        timestamp));
30    vm.stopPrank();
31 }
```

Recommended Mitigation:

Eliminate the swap incentive transfer completely as recommended in [H-1].

Medium Severity Findings

[M-1] Absence of `maxInputAmount` Slippage Protection in `swapExactOutput` Exposes Users to Extreme Capital Theft via Sandwich Attacks

Description:

In `TSwapPool.sol`, `swapExactOutput()` accepts only `inputToken`, `outputToken`, `outputAmount`, and `deadline`. It calculates `inputAmount` dynamically based on current pool reserves without a maximum bound.

Impact:

Traders have zero slippage protection. A malicious MEV searcher can sandwich the victim's transaction: 1. Front-run by swapping to severely deplete output reserves and inflate input reserves. 2. The victim's transaction executes at the manipulated exchange rate, consuming excessive input tokens. 3. Back-run by selling the output tokens back to the pool at the elevated price, extracting value directly from the victim.

Proof of Concept:

```
1 function
  test_PoC_SwapExactOutputMissingSlippageProtection_SandwichAttack
  () public {
2   poolToken.mint(user, 1000e18);
3   poolToken.mint(attacker, 1000e18);
4
5   // Initial expected cost for 5 WETH: ~52.79 Pool Tokens
6   uint256 expectedInputCost = pool.getInputAmountBasedOnOutput(
7     5e18,
8     poolToken.balanceOf(address(pool)),
9     weth.balanceOf(address(pool))
10  );
11
12  // Attacker front-runs by buying 40 WETH
13  vm.startPrank(attacker);
14  poolToken.approve(address(pool), type(uint256).max);
15  pool.swapExactOutput(poolToken, weth, 40e18, uint64(block.
    timestamp));
16  vm.stopPrank();
17
18  // User executes transaction without slippage bound
19  vm.startPrank(user);
20  poolToken.approve(address(pool), type(uint256).max);
21  uint256 userBalanceBefore = poolToken.balanceOf(user);
22  pool.swapExactOutput(poolToken, weth, 5e18, uint64(block.
    timestamp));
23  uint256 actualInputCost = userBalanceBefore - poolToken.
    balanceOf(user);
```

```
24     vm.stopPrank();
25
26     // User paid 700.89 Pool Tokens (>13x higher than initial
27         expected 52.79)
28     assertGt(actualInputCost, expectedInputCost * 3);
29 }
```

Recommended Mitigation:

Add a `maxInputAmount` parameter and revert if `inputAmount > maxInputAmount`:

```
1  --- a/src/TSwapPool.sol
2  +++ b/src/TSwapPool.sol
3  @@ -34,6 +34,7 @@ contract TSwapPool is ERC20 {
4      );
5      error TSwapPool__InvalidToken();
6      error TSwapPool__OutputTooLow(uint256 actual, uint256 min);
7  +    error TSwapPool__InputTooHigh(uint256 actual, uint256 max);
8      error TSwapPool__MustBeMoreThanZero();
9
10 @@ -340,6 +341,7 @@ contract TSwapPool is ERC20 {
11     IERC20 inputToken,
12     IERC20 outputToken,
13     uint256 outputAmount,
14  +    uint256 maxInputAmount,
15     uint64 deadline
16 )
17     public
18 @@ -355,6 +357,9 @@ contract TSwapPool is ERC20 {
19     inputReserves,
20     outputReserves
21 );
22  +    if (inputAmount > maxInputAmount) {
23  +        revert TSwapPool__InputTooHigh(inputAmount,
24  +    maxInputAmount);
25     }
26     _swap(inputToken, inputAmount, outputToken, outputAmount);
27 }
```

[M-2] Unused deadline Parameter in deposit Fails to Enforce Transaction Expiration, Enabling Execution of Stale Mempool Transactions

Description:

In `TSwapPool.sol`, `deposit()` accepts a `uint64 deadline` parameter. However, the `deadline` parameter is never used anywhere in the function body or modifier list. Un-

like `withdraw`, `swapExactInput`, and `swapExactOutput`, `deposit()` completely omits the `revertIfDeadlinePassed(deadline)` modifier.

The Solidity compiler explicitly flags this with:

```
1  Warning (5667): Unused function parameter. Remove or comment out
   the variable name to silence this warning.
2      --> src/TSwapPool.sol:117:9:
3          |
4  117 |         uint64 deadline
5          |         ~~~~~
```

Impact:

Because the `deadline` parameter is passed but never enforced, liquidity deposit transactions submitted with a deadline have no actual expiration. During network congestion, a deposit transaction submitted with a low gas fee can remain pending in the mempool indefinitely. Miners/validators or MEV bots can hold and execute the stale transaction hours or days later when pool reserve ratios and market prices have shifted substantially, inflicting unexpected slippage and severe impermanent loss on the liquidity provider.

Proof of Concept:

```
1  function test_PoC_DepositMissingDeadlineCheck() public {
2      vm.warp(100 days);
3      uint64 pastDeadline = uint64(block.timestamp - 1 days);
4
5      vm.startPrank(liquidityProvider);
6      weth.approve(address(pool), 10e18);
7      poolToken.approve(address(pool), 10e18);
8
9      // Succeeds despite deadline being 1 day in the past
10     pool.deposit(10e18, 0, 10e18, pastDeadline);
11     vm.stopPrank();
12 }
```

Recommended Mitigation:

Add the `revertIfDeadlinePassed(deadline)` modifier to the `deposit()` function declaration:

```
1  --- a/src/TSwapPool.sol
2  +++ b/src/TSwapPool.sol
3  @@ -118,6 +118,7 @@ contract TSwapPool is ERC20 {
4      )
5
6      external
7  +    revertIfZero(wethToDeposit)
8      +    revertIfDeadlinePassed(deadline)
9      returns (uint256 liquidityTokensToMint)
```

```
9      {
```

[M-3] Unassigned Named Return Variable in swapExactInput Always Evaluates to Zero, Breaking Composability

Description:

In `TSwapPool.sol`, `swapExactInput()` declares a named return value `returns (uint256 output)`. The function computes `outputAmount` as an internal local variable, but never assigns it to `output` or provides an explicit `return` statement.

Impact:

External contracts, aggregator routers, and keeper bots integrating with `TSwapPool.swapExactInput` receive 0 as the return value even when the trade successfully transfers tokens. Smart contract integrations asserting non-zero returns or tracking returned balances will revert, breaking composability with the broader DeFi ecosystem.

Proof of Concept:

```
1 function test_PoC_SwapExactInputReturnsZero() public {
2     vm.startPrank(user);
3     poolToken.approve(address(pool), 10e18);
4
5     uint256 returnedOutput = pool.swapExactInput(poolToken, 10e18,
6         weth, 0, uint64(block.timestamp));
7     // Output returns 0 despite valid swap execution
8     assertEq(returnedOutput, 0);
9     vm.stopPrank();
10 }
```

Recommended Mitigation:

Assign `output = outputAmount`; before function completion or return `outputAmount` directly:

```
1 --- a/src/TSwapPool.sol
2 +++ b/src/TSwapPool.sol
3 @@ -321,5 +321,6 @@ contract TSwapPool is ERC20 {
4         }
5
6         _swap(inputToken, inputAmount, outputToken, outputAmount);
7 +     output = outputAmount;
8     }
```

Low Severity Findings

[L-1] Swapped Arguments in LiquidityAdded Event Emission Corrupts Off-Chain Accounting

Description:

The `LiquidityAdded` event is declared as `(address indexed liquidityProvider, uint256 wethDeposited, uint256 poolTokensDeposited)`. However, in `_addLiquidityMintAndTransfer()`, it is emitted with parameter values inverted:

```
1 emit LiquidityAdded(msg.sender, poolTokensToDeposit, wethToDeposit);
```

Impact:

Off-chain indexing services (The Graph, Dune Analytics, DeFi Llama) and web3 user interfaces ingest transposed token amounts, reporting WETH deposits as Pool Tokens and vice versa.

Proof of Concept:

```
1 function test_PoC_LiquidityAddedEventSwappedParameters() public {
2     ERC20Mock asymmetricToken = new ERC20Mock();
3     TSwapPool asymPool = TSwapPool(factory.createPool(address(
4         asymmetricToken)));
5     weth.mint(liquidityProvider, 100e18);
6     asymmetricToken.mint(liquidityProvider, 200e18);
7
8     vm.startPrank(liquidityProvider);
9     weth.approve(address(asymPool), 100e18);
10    asymmetricToken.approve(address(asymPool), 200e18);
11
12    vm.expectEmit(true, false, false, true, address(asymPool));
13    emit LiquidityAdded(liquidityProvider, 200e18, 100e18);
14    asymPool.deposit(100e18, 100e18, 200e18, uint64(block.timestamp));
15    vm.stopPrank();
16 }
```

Recommended Mitigation:

Invert the argument order to match the event definition:

```
1 --- a/src/TSwapPool.sol
2 +++ b/src/TSwapPool.sol
3 @@ -193,7 +193,7 @@ contract TSwapPool is ERC20 {
4     uint256 liquidityTokensToMint
5     ) private {
6         _mint(msg.sender, liquidityTokensToMint);
```

```
7 -         emit LiquidityAdded(msg.sender, poolTokensToDeposit,
8 +         emit LiquidityAdded(msg.sender, wethToDeposit,
          poolTokensToDeposit);
9
10          // Interactions
```

[L-2] PoolFactory.createPool Permits WETH/WETH Pairs and Zero Address Token Deployments

Description:

PoolFactory.createPool() lacks validation to ensure `tokenAddress != address(0)` and `tokenAddress != i_wethToken`.

Impact:

Users can deploy useless pools paired with `address(0)` or duplicate pairs of `WETH/WETH`. Any interaction with a `WETH/WETH` pool attempts to execute with `inputToken == outputToken`, triggering `TSwapPool__InvalidToken()` reverts and cluttering factory state with defunct contracts.

Proof of Concept:

```
1 function test_PoC_PoolFactoryAllowsDuplicateWethPool() public {
2     address wethPool = factory.createPool(address(weth));
3     assertEq(factory.getPool(address(weth)), wethPool);
4 }
```

Recommended Mitigation:

Add validation checks rejecting zero address and WETH address:

```
1 --- a/src/PoolFactory.sol
2 +++ b/src/PoolFactory.sol
3 @@ -21,6 +21,8 @@ contract PoolFactory {
4     error PoolFactory__PoolAlreadyExists(address tokenAddress);
5     error PoolFactory__PoolDoesNotExist(address tokenAddress);
6 +   error PoolFactory__ZeroAddress();
7 +   error PoolFactory__CannotCreateWethPool();
8
9 @@ -47,6 +49,12 @@ contract PoolFactory {
10    function createPool(address tokenAddress) external returns (
11        address) {
12 +       if (tokenAddress == address(0)) {
```

```
12 +         revert PoolFactory__ZeroAddress();
13 +     }
14 +     if (tokenAddress == i_wethToken) {
15 +         revert PoolFactory__CannotCreateWethPool();
16 +     }
17 +     if (s_pools[tokenAddress] != address(0)) {
18 +         revert PoolFactory__PoolAlreadyExists(tokenAddress);
19 +     }
```

[L-3] PoolFactory.createPool Concatenates Token Name Instead of Symbol for LP Token Symbol

Description:

In `PoolFactory.sol`, the LP token symbol is defined as:

```
1 string memory liquidityTokenSymbol = string.concat("ts", IERC20(
    tokenAddress).name());
```

Impact:

For tokens with lengthy names (e.g. DAI whose name is "Dai Stablecoin" and symbol is "DAI"), the LP symbol becomes "tsDai Stablecoin" rather than "tsDAI", degrading user experience across wallets and block explorers.

Recommended Mitigation:

Use `.symbol()` instead of `.name()`:

```
1 --- a/src/PoolFactory.sol
2 +++ b/src/PoolFactory.sol
3 @@ -51,7 +51,7 @@ contract PoolFactory {
4     string memory liquidityTokenName = string.concat("T-Swap ",
5         , IERC20(tokenAddress).name());
6 -    string memory liquidityTokenSymbol = string.concat("ts",
7     IERC20(tokenAddress).name());
8 +    string memory liquidityTokenSymbol = string.concat("ts",
9     IERC20(tokenAddress).symbol());
```

Informational Findings

[I-1] Missing Zero Address Validation in PoolFactory Constructor Risks Permanent Misconfiguration

Description:

The constructor in `PoolFactory.sol` sets `i_wethToken = wethToken` without checking `if (wethToken == address(0))`. If deployed with `address(0)`, every pool deployed by the factory will pair against `address(0)`, rendering the factory unusable.

Recommended Mitigation:

Add zero address check in constructor:

```
1 constructor(address wethToken) {
2     if (wethToken == address(0)) {
3         revert PoolFactory__ZeroAddress();
4     }
5     i_wethToken = wethToken;
6 }
```

[I-2] Unused Custom Error PoolFactory__PoolDoesNotExist Increases Bytecode Size

Description:

`PoolFactory.sol` declares `error PoolFactory__PoolDoesNotExist(address tokenAddress)`; at line 22, but this error is never referenced or thrown anywhere in the contract.

Recommended Mitigation:

Remove the unused custom error from `PoolFactory.sol`.

[I-3] Unused Storage Read poolTokenReserves in deposit Wastes Gas (Compiler Warning 2072)

Description:

In `TSwapPool.sol` line 131:

```
1 uint256 poolTokenReserves = i_poolToken.balanceOf(address(this));
```

This local variable is never read or used in any subsequent lines of `deposit()`, triggering Solc compiler warning `Warning (2072): Unused local variable` and wasting gas on an unnecessary external call.

Recommended Mitigation:

Delete line 131 from `TSwapPool.sol`.

[I-4] Unindexed Event Parameters in PoolFactory.PoolCreated and TSwapPool.Swap Prevent Efficient Log Filtering**Description:**

Events should index up to 3 address/topic parameters to allow lightweight filtering and querying by off-chain indexers. Several events in the protocol omit the `indexed` keyword on critical address parameters:

1. `PoolFactory.PoolCreated`: Neither `tokenAddress` nor `poolAddress` is indexed.
2. `TSwapPool.Swap`: `tokenIn` and `tokenOut` addresses are not indexed.

Recommended Mitigation:

Add `indexed` keyword to address parameters in `PoolFactory.sol` and `TSwapPool.sol`.

[I-5] Strict Equality Comparison in revertIfZero Modifier**Description:**

`TSwapPool.sol` modifier `revertIfZero` uses a strict equality check `if (amount == 0)` flagged by Slither (`incorrect-equality`).

Recommended Mitigation:

Document strict equality intentionality or use defensive comparison `amount <= 0`.

[I-6] Erroneous Mathematical Explanation in NatSpec / Comments Misleads Developers**Description:**

In `TSwapPool.sol` lines 132-143, inline comments claim $weth/poolTokens = k$. In Constant Product AMMs ($x \cdot y = k$), the invariant is multiplication, NOT division ($x/y = k$). The ratio of reserves represents instantaneous spot price, not the invariant k .

Recommended Mitigation:

Update comments to correctly describe the constant product invariant $x \cdot y = k$.

[I-7] Redundant Wrapper Function `totalLiquidityTokenSupply()` Adds Bytecode Overhead**Description:**

`TSwapPool.sol` defines a wrapper function `totalLiquidityTokenSupply()` that simply returns `totalSupply()`. Because `TSwapPool` inherits from OpenZeppelin's `ERC20`, `totalSupply()` is already public.

Recommended Mitigation:

Remove `totalLiquidityTokenSupply()` and call `totalSupply()` directly.

[I-8] Unnamed Magic Numbers Obscure Business Logic and Contribute to Math Errors**Description:**

Across `TSwapPool.sol`, numeric literals such as 997, 1000, 10000, 1_000_000_000_000_000_000, and 1e18 are hardcoded directly into arithmetic expressions without constant declarations, directly contributing to finding [H-2].

Recommended Mitigation:

Define named constants such as `FEE_DENOMINATOR = 1000;`, `FEE_NUMERATOR = 997;`, and `ONE_TOKEN = 1e18;`.

[I-9] Private Function Declarations Preceding Public View Functions Violates Solidity Style Guide

Description:

In `TSwapPool.sol`, private functions `_swap()` and `_isUnknown()` are placed before public view functions, violating Solidity Style Guide layout conventions.

Recommended Mitigation:

Reorder functions according to standard visibility ordering (external, public, internal, private).

[I-10] License Identifier Discrepancy Between Protocol (GPL-3.0) and Dependencies (MIT)

Description:

`PoolFactory.sol` and `TSwapPool.sol` specify `// SPDX-License-Identifier: GNU General Public License v3.0`, whereas imported OpenZeppelin contracts use `MIT`. GPL-3.0 has strong reciprocal copyleft terms that may impose licensing constraints on downstream integrators.

Recommended Mitigation:

Confirm that GPL-3.0 aligns with the protocol distribution roadmap.

[I-11] Unused Function Parameter `deadline` in `deposit` Triggers Compiler Warning (5667)

Description:

In `TSwapPool.sol` line 117, `deposit()` accepts `uint64 deadline` as a parameter, but the parameter is never referenced or checked anywhere within the function body or modifier list, triggering Solc compiler warning `Warning (5667): Unused function parameter`.

(Note: For the security impact of missing deadline enforcement against stale mempool transactions, see finding [M-2]).

Recommended Mitigation:

Add the `revertIfDeadlinePassed(deadline)` modifier to enforce expiration (as described in [M-2]), or remove the parameter if deadline checking is not desired.

Test Suite Quality & Mutation Testing Review

Following Phase 5.4 of The Tincho Method, an **Automated Mutation Testing Runner** (`scripts/mutation_runner.py`) was executed against the codebase to evaluate test sensitivity:

$$\text{Mutation Score} = \left(\frac{\text{Killed Mutants}}{\text{Total Mutants}} \right) \times 100\%$$

- **Total Tactical Mutants:** 16 (operator inversions, math alterations, fee bypasses, boundary deletions)
- **Original Client Test Suite Score: 31.2%** (Only 5 of 16 mutants killed; **11 Mutants Survived!**)
- **Combined Suite with Audit PoCs Score: 100.0%** (16 of 16 mutants killed)

Surviving mutants in the original test suite revealed that fee calculations, deadline checks, and deposit/withdrawal slippage boundaries were completely untested by the client test suite.

General Recommendations

1. **Adopt FREI-PI and CEI Standards:** Complete all state balance updates before making external token transfers. Never introduce side-effect token giveaways inside core AMM swap paths.
2. **Decimal Standardization:** Ensure tokens with decimals other than 18 (e.g. USDC with 6 decimals) are handled with proper decimal scaling or validated upon pool creation.
3. **Incorporate ReentrancyGuard:** Guard all external entrypoints (`deposit`, `withdraw`, `swapExactInput`, `swapExactOutput`) with OpenZeppelin's `ReentrancyGuard` to prevent reentrancy via ERC777 tokens.
4. **Continuous Stateful Invariant Testing:** Maintain Foundry invariant fuzz suites testing constant product maintenance, solvency, and token ratios across thousands of randomized interaction sequences in CI/CD pipelines.